School of Computing Science & Engineering & Artificial Intelligence (SCAI)

Reinforcement Learning

Lab File

Reinforcement Learning (CSA4029)
Slot: B22+E14+E22

Semester – Winter Semester 2026-2027

Submitted to: **Dr. Manoj Kumar**

Submitted by: **Meera Singh (23BAI11317)**

Acknowledgement

I would like to express my sincere gratitude to all those who provided invaluable support and guidance throughout the completion of this **Reinforcement Lab Manual.**

My deepest appreciation goes to **Manoj Kumar Sir**, whose exceptional mentorship, insightful instructions, and unwavering encouragement were crucial in navigating the complexities of AWS services and successfully completing each lab experiment. Sir's dedication to hands-on learning significantly enhanced my understanding and practical skills in cloud computing.

I am also thankful to my family and friends for their continuous support and for providing a conducive environment for my studies and practical work.

Finally, I extend my thanks to the **VIT BHOPAL UNIVERSITY** for providing the necessary resources and platform to undertake these practical experiments, which are vital for a comprehensive understanding of cloud technologies.

This lab manual represents a significant step in my practical understanding of AWS, and I am grateful for the opportunity to gain this hands-on experience.

Experiment 2: SARSA-Lambda

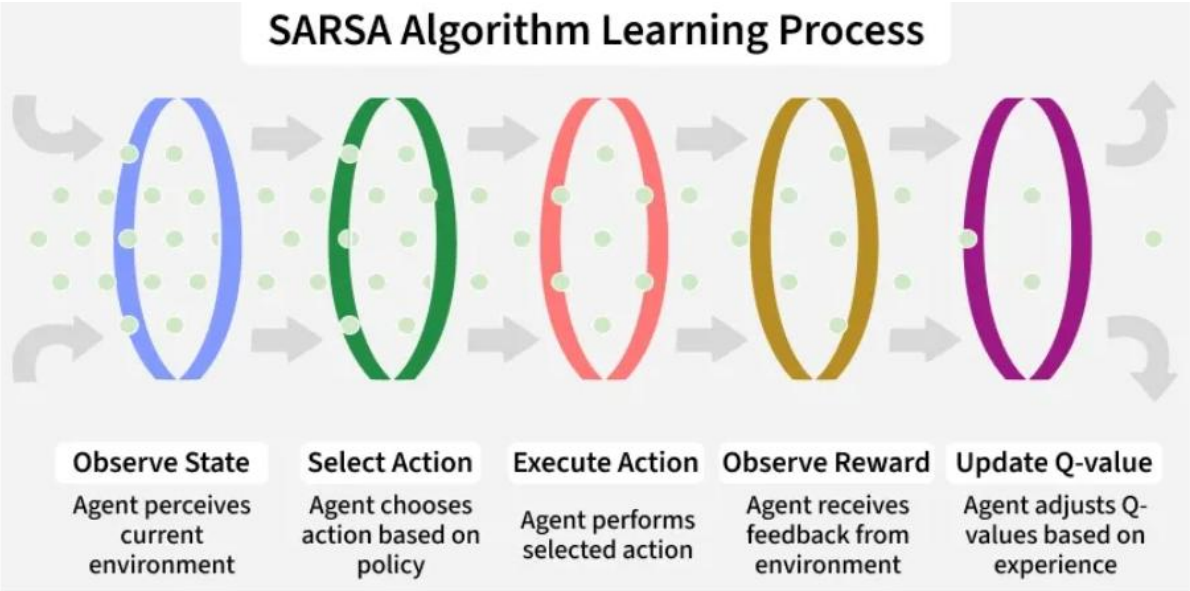
Aim:-

To implement SARSA-Lambda algorithm using Python**.**

Theory:-

SARSA is an on-policy reinforcement learning algorithm. SARSA-Lambda introduces eligibility traces to improve learning speed.

Update rule: $Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta\, e(s,a)$

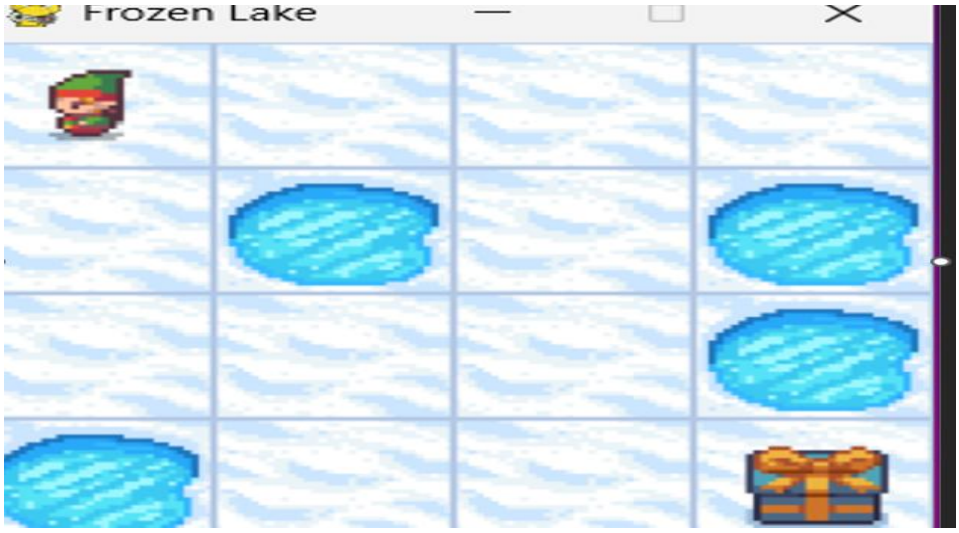


SARSA algorithm Learning Process

Code:-

```
import numpy as np
import gymnasium as gym
import time

env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="human")

q_table = np.zeros((env.observation_space.n, env.action_space.n))
eligibility = np.zeros_like(q_table)

alpha = 0.1
gamma = 0.9
lambda_ = 0.8
epsilon = 0.1
episodes = 500

# Training
```

```
for episode in range(episodes):
    state, _ = env.reset()

    if np.random.uniform(0,1) < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(q_table[state])

    eligibility *= 0
    done = False

    while not done:
        next_state, reward, terminated, truncated, _ = env.step(action)

        if np.random.uniform(0,1) < epsilon:
            next_action = env.action_space.sample()
        else:
            next_action = np.argmax(q_table[next_state])

        td_error = reward + gamma * q_table[next_state, next_action] -
q_table[state, action]

        eligibility[state, action] += 1

        q_table += alpha * td_error * eligibility
        eligibility *= gamma * lambda_

        state = next_state
        action = next_action

        done = terminated or truncated

print("Training Finished")

# Simulation
print("Simulation Starting...")
state, _ = env.reset()
done = False

while not done:
    env.render()
    time.sleep(0.5)

    action = np.argmax(q_table[state])
    state, reward, terminated, truncated, _ = env.step(action)

    done = terminated or truncated
```

```
env.render()
env.close()
```

Output:-